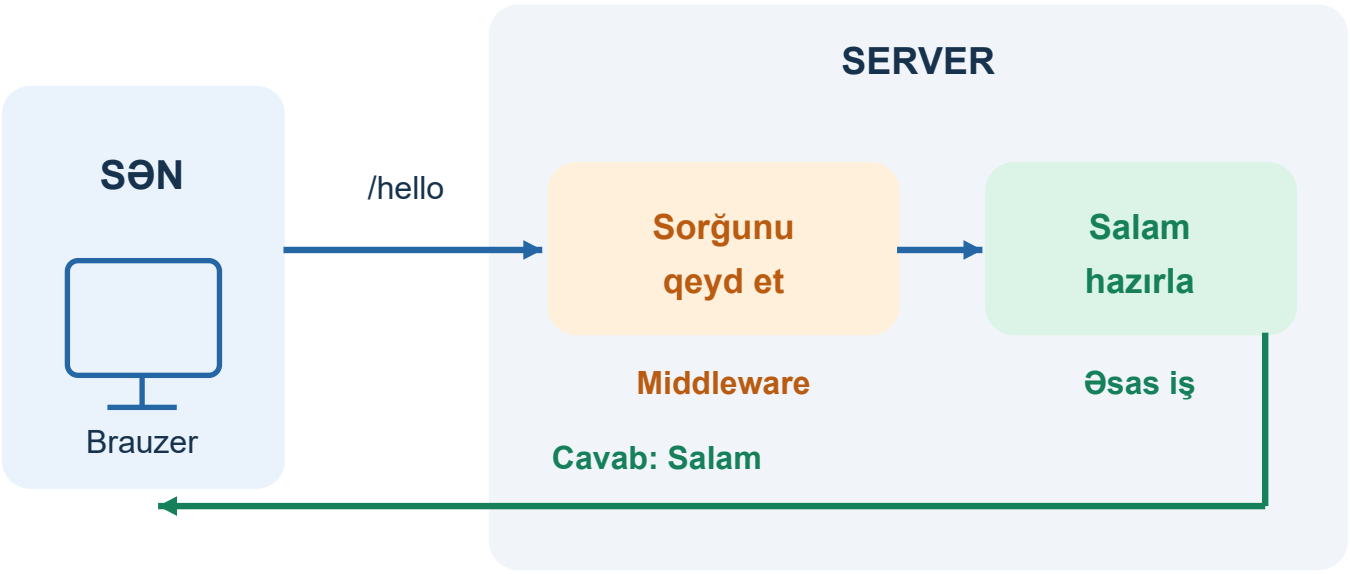
1. Əvvəl bir sorğunu təsəvvür et

Sən brauzerdə **/hello** açırsan. Brauzer serverə “bu ünvanı aç” deyir. Server cavab hazırlayan proqramdır. Bu nümunədə cavabımız sadəcə **Salam** olacaq.



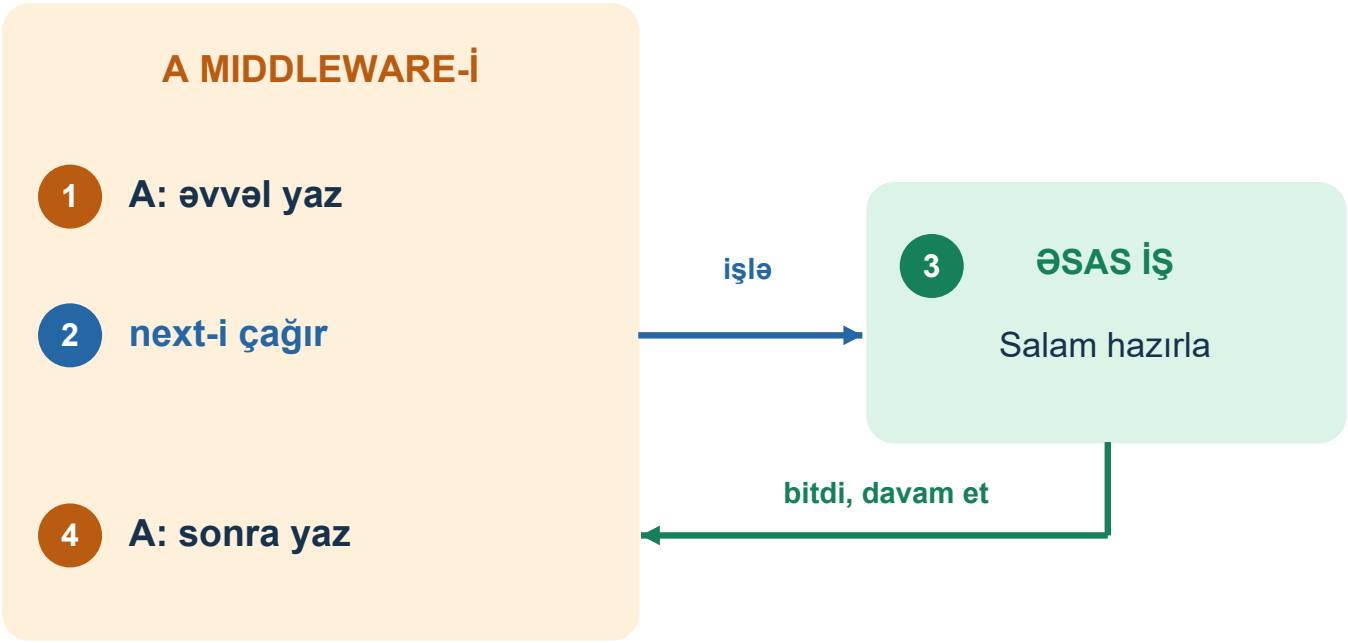
Narıncı hissəyə bax: əsas kod Salam hazırlamamışdan əvvəl sorğunun gəldiyini qeyd edir. **Bu ayrıca mərhələyə middleware deyirik.**

Niyə ayrıca yazırıq? Çünki sabah başqa ünvanlar da əlavə edəndə eyni qeyd etmə işini hər birinin kodunda yenidən yazmaq istəmirik.

İlk fikir: middleware sorğuya çatanda ayrıca iş görür, sonra əsas işə keçməyə icazə verə bilər.

2. next nə deməkdir?

next burada “məndən sonrakı hissəni işlət” deməkdir. A öz işinin bir hissəsini görür, davamını çağırır, o bitəndə qalan işini görür.



A, əsas iş bitənə qədər gözləyir.

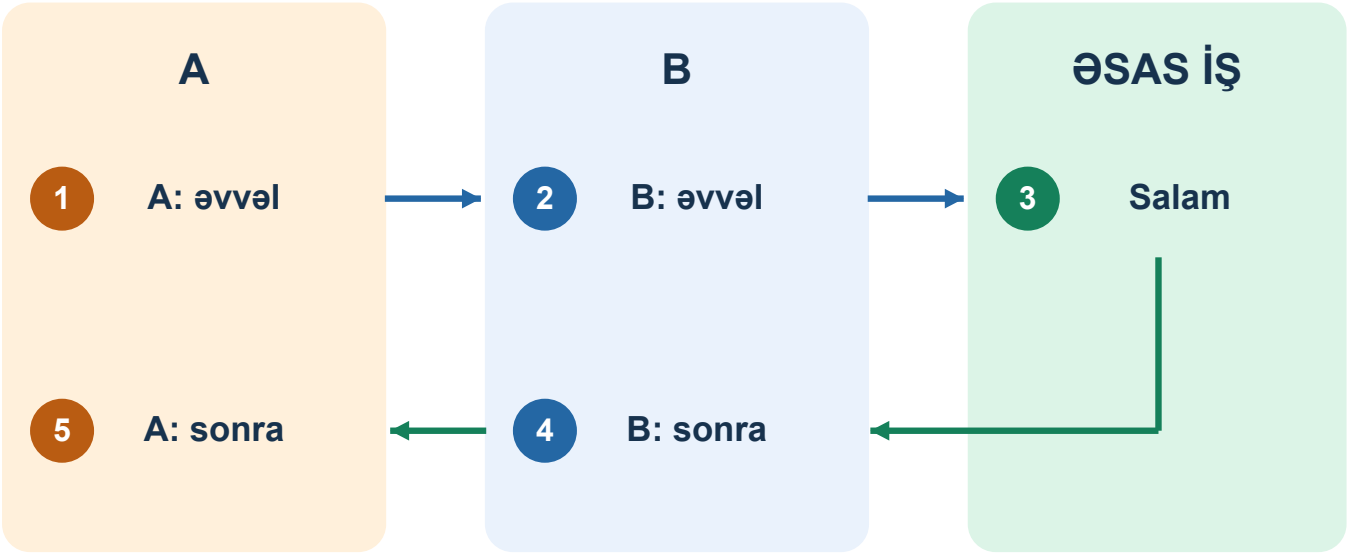
```
Console.WriteLine("A: evvel"); // 1
await next(context);           // 2: esas isi gozle
Console.WriteLine("A: sonra"); // 4
```

Şəkildəki **3-cü addımı əsas kod** edir: Salam hazırlayır. O bitəndən sonra A 4-cü addımdan davam edir. A yenidən əvvəldən başlamır.

await çağırılan işin tamamlanmasını gözləmək üçün yazılır. **context** isə hazırkı sorğu və cavabın məlumatlarını saxlayan obyektdir. Hələlik kodda bu ikisini belə oxu: “davamını işlət və bitməsini gözlə”.

3. İki middleware olanda nə olur?

Birinci mərhələni A, ikincini B adlandıraq. Onların qoşulduğu sıra **A, sonra B**-dir. Bu mərhələlər yoluna **pipeline** deyirik.



Əvvəl: A → B → əsas iş. Sonra: B → A.

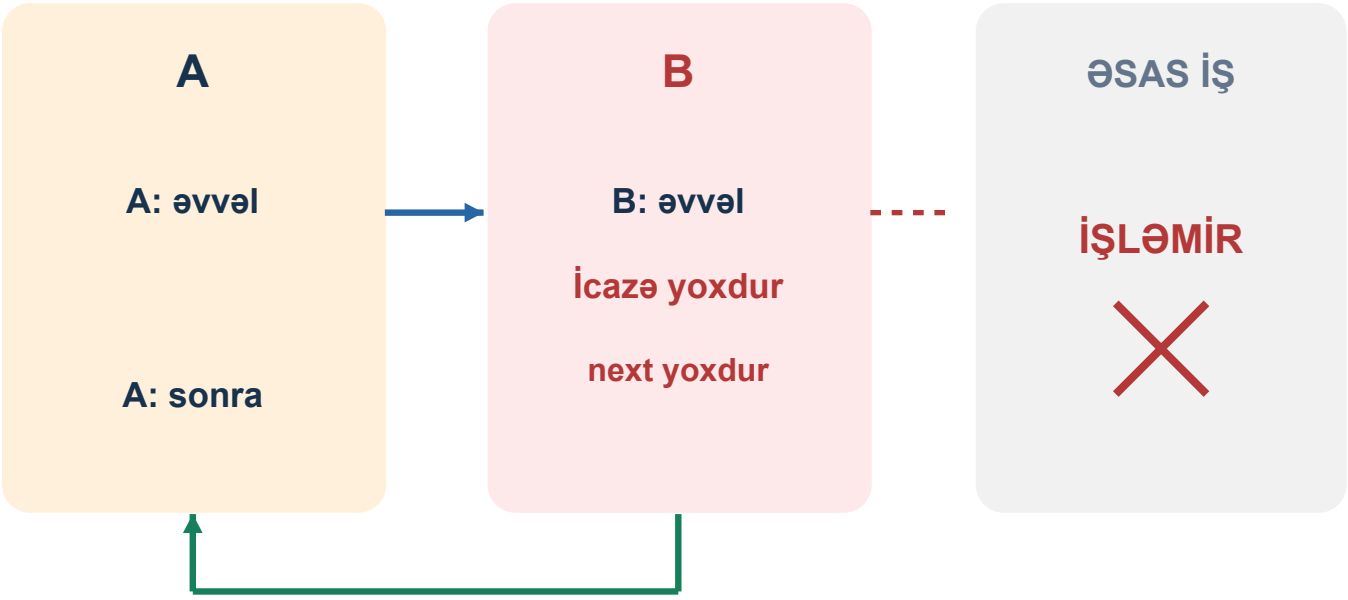
```
1  A: evvel
2  B: evvel
3  Salam hazirlanir
4  B: sonra
5  A: sonra
```

A, B-ni çağırıb gözləyir. B də əsas işi çağırıb gözləyir. Əsas iş bitəndə əvvəl **B öz qalan sətirini**, sonra **A öz qalan sətrini** işlədir.

Bu rəqəmlər **işləmə sırasıdır**. Ayrı bir “geri sorğu” yaranmır. Gözləyən funksiyalar çağırdıqları iş bitəndə davam edir.

4. Middleware davamı dayandıra bilər

Tutaq ki, B bu ünvanı açmağa icazə vermir. B özü “İcazə yoxdur” cavabı yazır və **next çağırmır.**



B cavab verir. Onu gözləyən A isə davam edir.

```
context.Response.StatusCode = 403;
await context.Response.WriteAsync("Icaze yoxdur");
return;
```

403 bu nümunədə icazə verilmədiyini bildirən cavab kodudur. **WriteAsync** cavab mətnini yazır. **return** B-nin həmin funksiyasından çıxır.

Əsas işə çatmırıq, çünki B onu çağırmayıb. Amma A əvvəl B-ni çağırmışdı; B bitəndə A qalan hissəsinə davam edir. Çıxış: **A əvvəl → B əvvəl → A sonra.**

Davamı belə kəsməyin texniki adı **short-circuit**-dir. Yəni “qalan hissəni işlətmədən burada cavablandır”.

5. Şəkildə gördüyümüzün tam kodu

Boş ASP.NET Core layihəsində Program.cs-yə bunu yazmaq olar. **Use** A mərhələsini qoşur. **MapGet** /hello açılarkən işləyəcək əsas kodu qeyd edir.

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

app.Use(async (context, next) =>
{
    Console.WriteLine("A: evvel");
    await next(context);
    Console.WriteLine("A: sonra");
});

app.MapGet("/hello", () =>
{
    Console.WriteLine("Salam hazirlanir");
    return "Salam";
});

app.Run();
```

Əsas kodun texniki adı endpoint-dir. Burada endpoint “Salam” qaytaran hissədir. GET məlumat istəmək üçün HTTP sorğu növüdür. HTTP brauzer ilə serverin ünsiyyət qaydalarıdır.

async gözləmə olan bu funksiyanı işarələyir. Parametrsiz **app.Run()** isə tətbiqi işə salır. /hello açıldıqda brauzer Salam görür; terminalda **A əvvəl, Salam hazırlanır, A sonra** görünür.

6. Real layihədə bu mərhələlər nə edir?

Sən /orders açıb sifarişləri görmək istəyirsən. Əsas kod sifarişləri hazırlayır. Ondan əvvəl ümumi yoxlamalar ola bilər:

1. Yolu tap
Bu ünvanı hansı kod cavablandıracaq? Buna routing deyilir.

2. İstifadəçini tanı
Sorğunu göndərən kimdir? Buna authentication deyilir.

3. İcazəni yoxla
Bu adam sifarişləri görə bilər? Buna authorization deyilir.

4. Əsas işi gör
İcazə varsa sifarişləri hazırla və cavab ver.

Niyə sıra vacibdir? İcazəni istifadəçinin kimliyinə görə yoxlayırsansa, əvvəl kim olduğunu müəyyən etməlisən. İcazə yoxdursa davam kəsilir və sifariş koduna çatılmır.

Use: davam edə bilən mərhələ əlavə edir.
Run(context => ...): next almayan sonlandırıcı işləyicidir.
Map: müəyyən yol üçün ayrıca qol ayırır.
UseWhen: şərtə uyğun qol yaradır; davam kəsilməsə əsas yola qoşula bilər.

Bu adlar tətbiqin yolunu qurmaq üçündür. Hazır middleware-lər də var; bütün yoxlamaları özün sıfırdan yazmalı deyilsən.

7. Müsahibə üçün qısa cavablar

Middleware nədir? Sorğu və cavab üzərində işləyən mərhələdir. Davamından əvvəl və sonra iş görə, ya da davamı kəsə bilər.

Pipeline nədir? Middleware-lərin qoşulduğu işləmə ardıcıllığıdır.

next nə edir? Qalan hissəni çağırır. await ilə gözləyəndə o bitdikdən sonra indiki funksiyanın növbəti sətri işləyir.

next-dən sonrakı kod həmişə işləyir? Davamda tutulmayan xəta atılarsa adi sətirə keçilməyə bilər. Həm uğurlu, həm xətalı çıxışda iş görmək üçün try/finally istifadə edilir.

Xəta middleware-i niyə əvvəl olur? Özündən sonrakı hissəni çağırarkən baş verən xətanı tuta bilsin deyə. catch xətanı tutur; finally isə çıxış işini görür.

Cavabın başlığını sonra dəyişmək olar? Başlıq Content-Type kimi əlavə cavab məlumatıdır. Cavab artıq başlayıbsa onu dəyişmək mümkün olmaya bilər. Response.HasStarted bunu göstərir.

Middleware hər sorğuda yenidən yaranır? Adi siniflə yazılan middleware pipeline qurulanda yaradılır; işləyici metodu hər uyğun sorğuda çağırılır. Sorğu məlumatını ortaq sahədə saxlamaq olmaz.

Scoped servisi necə alırıq? Scoped servis adi HTTP sorğusu daxilində paylaşılır. Adi middleware-də InvokeAsync parametrindən alınır. IMiddleware üsulunda scoped qeydiyyatla konstruktordan da alına bilər.

Filter ilə fərqi nədir? Middleware ümumi HTTP yolunda, MVC filter isə controller-in konkret işləyici metodunun icra mərhələlərində işləyir.

Rəsmi istinadlar: middleware, sinif və DI, IMiddleware, filters.